



Clases abstractas e interfaces: primero el contrato

El problema

La clase anterior dejó armada la jerarquía de cobros del local. Arriba, la clase Pago con el número de operación, la venta y el encabezado del comprobante. Abajo, PagoDebito, PagoCredito y PagoQR, cada uno con lo suyo. Lo común quedó escrito una sola vez. Pero quedaron dos agujeros.

El primer problema es que esta línea compila:

```
Pago pago = new Pago(1, venta);
```

Ese objeto no representa nada. En el local no se cobra “con pago”: se cobra con débito, con crédito o con QR. Pago se escribió para ser heredada, no para ser instanciada, y no vimos cómo dejar eso explícito.

El segundo problema podría aparecer con un requerimiento nuevo. Por ejemplo, supongamos que pagar con tarjeta (débito o crédito) y una billetera virtual aplican un impuesto distinto. Ese cálculo dependería del medio de pago: el débito retiene 0,8%, el crédito bastante más, el QR tiene su propia comisión. Pago no puede calcularlo, no sabe de esos impuestos. Cada subclase sí.

Pero entonces, ¿cómo hace el resto del sistema para pedirle a un pago cualquiera cuánto se acredita? Si Pago no declara ese método, no se lo puede llamar. Y si Pago lo declara con un cuerpo vacío, la subclase que se olvide de sobreescribirlo va a heredar un cálculo erróneo y que falla silenciosamente.

Hace falta poder decir “esta clase está incompleta a propósito, y estas son las partes que quien la herede tiene que completar”.



Clase declarada como abstract (incompleta a propósito)

```
public abstract class Pago {  
  
    private final Integer numeroOperacion;  
    private final Venta venta;  
  
    public Pago(Integer numeroOperacion, Venta venta) {  
        if (venta == null) {  
            throw new IllegalArgumentException("La venta es obligatoria");  
        }  
        this.numeroOperacion = numeroOperacion;  
        this.venta = venta;  
    }  
  
    public Double getMonto() {  
        return this.venta.getTotal();  
    }  
  
    // Método ABSTRACTO: cada medio de pago aplica un % distinto,  
    // Pago no puede calcularlo. Solo declara que el cálculo existe.  
    public abstract Double montoAcreditado();  
  
    // Metodo CONCRETO: el encabezado del comprobante es igual para todos.  
    public void mostrarComprobante() {  
        System.out.println("Operacion nro " + this.numeroOperacion);  
        System.out.println("Detalle: " + this.venta.getDescripcion());  
        System.out.println("Monto: $" + this.getMonto());  
        System.out.println("Se acreditan: $" + this.montoAcreditado());  
    }  
}
```

Con esas dos abstract cambian dos cosas. `new Pago(1, venta)` ya no compila, el error es directo: `Pago is abstract; cannot be instantiated`. Y toda subclase que quiera instanciarse queda obligada a escribir `montoAcreditado()`.

Notar la última línea de `mostrarComprobante()`. Es un método concreto, escrito en `Pago`, que llama a un método abstracto. Funciona porque en tiempo de ejecución el objeto siempre es un `PagoDebito` o un `PagoQR`, una clase que sí lo implementó (le dio cuerpo al método abstracto). La clase abstracta escribe el flujo general y deja huecos con nombre para que los complete cada especialización (cada clase hija).

Las reglas a recordar:

- Una clase con al menos un método abstracto tiene que declararse `abstract`.
- Al revés no: una clase puede ser `abstract` sin tener ningún método abstracto, simplemente para prohibir que se la instancie.
- Una clase abstracta sí tiene constructor. No se lo llama con `new`, se lo llama con `super(...)` desde el constructor del hijo, igual que antes.
- Una subclase que no implementa todos los métodos abstractos que hereda tiene que declararse `abstract` ella también, y quedar a la espera de que otra los complete.



Lo que la herencia no alcanza a modelar

El local suma dos requerimientos. Ahora también se cobra en efectivo, y hay que poder anular una operación del día.

El efectivo entra sin problema en la jerarquía: PagoEfectivo es un Pago, y su `montoAcreditado()` devuelve el total, porque nadie aplica una comisión. La anulación es otra cosa. Un pago con débito se anula pidiéndole el reintegro a la tarjeta. Un pago con QR se anula avisando a la billetera virtual. Un pago en efectivo no se anula desde el sistema, se abre la caja y se devuelve la plata.

La tentación es poner `anular()` en Pago. Pero entonces PagoEfectivo hereda una capacidad que no tiene, y la clase queda mintiendo sobre lo que sabe hacer.

La segunda tentación es meter una clase intermedia, PagoElectronico, entre Pago y los medios que sí se anulan. Y funcionaría, hasta que aparece el dato de que una Venta también se anula. Venta no es un Pago, no comparte ni un ancestro con la jerarquía de cobros, y Java no deja heredar de dos clases a la vez.

El problema es que “poder anularse” no describe qué es un objeto. Describe algo que el objeto sabe hacer. Alcanza a algunas subclases de Pago y también a clases que no tienen nada que ver con Pago.

interface

Para eso está la interfaz. Casi toda la bibliografía utiliza la analogía de “contrato” para definirla. En esta se declara un conjunto de métodos sin escribir la implementación, y cualquier clase que implemente este “contrato”, esté donde esté, puede comprometerse a cumplirlo.

```
public interface Anulable {  
    // El plazo que da la tarjeta para revertir una operacion.  
    int DIAS_PARA_ANULAR = 30;  
  
    // Devuelve true si la anulacion se pudo hacer.  
  
    boolean anular();  
  
    boolean estaAnulado();  
}
```

Los métodos no tienen cuerpo, solo firma y punto y coma. Tampoco dice `public`, aunque lo son: en una interfaz todos los métodos son públicos y abstractos, el compilador lo completa. Y tiene sentido, un contrato con cláusulas privadas no le sirve a nadie, la interfaz existe precisamente para exponer lo que un objeto le ofrece al resto del sistema.

Una interfaz también puede declarar constantes, y ahí se ahorra todavía más sintaxis: `DIAS_PARA_ANULAR` es `public static final` aunque no aparezca ninguna de las tres palabras. Que sea `static` tiene la misma razón que en la clase 4, el valor es uno solo y no pertenece a ningún objeto en particular. Que sea `final` es lo que la vuelve legítima acá: una interfaz no guarda estado, y un valor que no cambia nunca no es estado, es parte del contrato. Quien



implementa la interfaz la escribe por su nombre a secas, y quien solo conoce el contrato la escribe Anulable.DIAS_PARA_ANULAR.

Una clase se compromete con implements, y puede hacerlo además de heredar:

```
public class PagoQR extends Pago implements Anulable {

    private final String billetera;
    private boolean anulado;

    // ... constructor de la clase anterior, con su super(...) ...

    @Override
    public double montoAcreditado() {
        return this.getMonto() * (1 - ARANCEL_QR);
    }

    @Override
    public boolean anular() {
        if (this.anulado || this.diasDesdeElCobro() > DIAS_PARA_ANULAR) {
            return false;
        }
        // ... aviso a la API de la billetera ...
        this.anulado = true;
        return true;
    }

    @Override
    public boolean estaAnulado() {
        return this.anulado;
    }
}
```

PagoQR es un Pago y sabe anularse. PagoEfectivo es un Pago y no implementa Anulable, así que ni siquiera se le puede pedir. Y Venta, que vive en otra parte del sistema, escribe public class Venta implements Anulable y cumple el mismo contrato con su propia implementación. Nadie hereda nada de nadie.

Ahí está la ganancia sobre la herencia simple: una clase tiene un solo padre, pero puede implementar muchas interfaces (cumplir todos los contratos) las que haga falta, separados por coma.

```
public class PagoDebito extends Pago implements Anulable, Exportable {
```

Ese PagoDebito es un Pago, sabe anularse y además sabe volcarse a la línea de texto que pide el archivo del cierre de caja, que es lo que exige el contrato Exportable. Tres compromisos, una sola herencia.

Dos detalles más. Una interfaz no se instancia y no tiene constructor ni atributos de instancia, porque no hay estado que guardar. Desde Java 8 se le pueden agregar métodos con cuerpo marcados como default, un recurso pensado para agregar métodos a interfaces ya publicadas sin romper a quienes las implementan. Conviene saber que existe, pero para modelar contratos alcanza con la forma clásica.



Cuál de los dos

Las dos herramientas obligan a las subclases a completar métodos, y las dos evitan que se instancie algo incompleto. La pregunta que las separa no es técnica, es de significado, la misma que decidía si correspondía heredar.

	Clase abstracta	Interfaz
La relación	“es un”	“sabe hacer”, “es capaz de”
Ejemplo	Un pago con QR es un pago.	Un pago con QR sabe anularse.
En Java	<code>extends Pago</code>	<code>implements Anulable</code>
Cuántas	Una sola por clase.	Todas las que haga falta.
Qué aporta	Atributos y código ya escrito, más los huecos obligatorios.	Solo el contrato, ninguna implementación.
Cuándo conviene	Varias clases muy parecidas comparten estado y código.	Una capacidad que aparece en clases sin parentesco.

La regla práctica: si al escribir la frase “es un” suena natural y además hay código para compartir, clase abstracta. Si lo que se repite es una capacidad suelta que atraviesa la jerarquía, interfaz.

Programar contra la interfaz

El beneficio de todo esto no se ve en la clase que implementa, se ve en el código que la usa. La biblioteca de Java está construida así, y el caso más visible es el de las colecciones.

```
List<String> nombres = new ArrayList<>();
```

List es una interfaz: el contrato de una secuencia ordenada que admite repetidos, con add, get, size y el resto. ArrayList es una de sus implementaciones. La variable se declara con el tipo del contrato y solo la línea del new menciona la implementación concreta. El día que convenga cambiar a LinkedList, se cambia esa línea y nada más, porque el resto del código nunca dependió de otra cosa que del contrato.

Lo mismo vale para el código propio. Un método declarado así:

```
public static int anularTodo(Anulable[] operaciones) { ... }
```

recorre pagos con QR, pagos con débito y ventas sin distinguirlos, sin saber cómo anula cada uno y sin tener que tocarse cuando el año que viene aparezca un medio de pago nuevo. Le alcanza con que cumpla el contrato.

Que un mismo llamado a `anular()` termine ejecutando código distinto según quién esté del otro lado es el mecanismo que sostiene todo lo anterior. Tiene nombre propio, polimorfismo, y es el tema de la clase que viene.

Bibliografía sugerida para profundizar

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.



- Capítulo 10: Object-Oriented Programming: Polymorphism and Interfaces. Trata las clases abstractas y las interfaces juntas y en ese orden, con el mismo argumento de esta clase: la superclase declara lo que las subclasses tienen que completar.

Aguilar, L. J. (2011). *Programación en Java*. McGraw-Hill.

- Capítulo 12, sección 12.1 (p. 303): interfaces, su implementación y su función como alternativa a la herencia múltiple.
- Capítulo 12, sección 12.3 (p. 308): clases abstractas, y su desarrollo completo en el capítulo 14, sección 14.2 (p. 344).

García Llinás, L. F. (2010). *Todo lo básico que debería saber sobre Programación orientada a objetos en Java*. Ediciones Uninorte.

- Capítulo 1, sección 1.11 (p. 96): clases abstractas, con consideraciones prácticas en las páginas 105 y 106.
- Capítulo 1, sección 1.16 (p. 124): definición e implementación de interfaces.